3. Actividad seguridad

Conceptos fundamentales para proteger sistemas y datos en el mundo digital

Conceptos Fundamentales

Los pilares de la seguridad informática que todo desarrollador debe conocer

Permissions

Reglas que controlan qué puede hacer un usuario en el sistema. Define el acceso a recursos como archivos, APIs y sistemas.

Ejemplo: Un usuario puede leer datos pero no eliminarlos

Password Segura

Contraseña difícil de adivinar que debe tener longitud, complejidad y no ser reutilizada en otros sitios.

Ejemplo: G7!k9@pL2# en vez de 123456

Vulnerabilidad

Debilidad en un sistema que puede ser explotada por un atacante para comprometer la seguridad.

Ejemplo: Un endpoint sin protección que expone datos

Intrusión

Acceso no autorizado a un sistema usando vulnerabilidades para entrar sin permiso.

Ejemplo: Un hacker accede a una base de datos.

Seguridad de red:

Protección de la comunicación y sistemas conectados.

Ejemplo: Bloquear acceso externo a una base de datos.

Made with GAMMA

Autenticación vs Autorización

Autenticación 🔑

Verificar quién eres. El sistema valida tu identidad, normalmente con usuario + contraseña.

Ejemplo: Ingresas email y password para entrar al sistema

Autorización ⚖️

Verificar qué puedes hacer. Ocurre después de autenticarte y define permisos dentro del sistema.

Ejemplo: Puedes ver datos, pero no eliminarlos

Diferencia clave:

- Autenticación es identidad (quién eres)
- Autorización son permisos (qué puedes hacer).

Primero te identificas, luego el sistema decide tus permisos.

Made with GAMMA

Roles (Admin/User) y APIs

Es un sistema de autorización que define niveles de acceso.

Sistema de Roles

Admin

Crea, edita y elimina información

User

Solo puede ver información

Relación con APIs

1. Login → obtienes token ("llave digital")
2. Llamas a API (/users)
3. Servidor valida:

- Quién eres (autenticación)
- Qué puedes hacer (autorización)

Made with GAMMA

Passwords Seguras

Qué las hace seguras

- Longitud suficiente (mínimo 12 caracteres)
- Mayúsculas, minúsculas, números y símbolos
- No reutilizadas en otros sitios
- No guardadas en texto plano

01

Usuario ingresa contraseña

En el formulario de login

02

Backend la convierte en hash

Transformación irreversible que crea una "huella digital"

03

Se guarda el hash

En la base de datos, nunca la contraseña real

04

En login: se compara

Se hace hash otra vez y se compara con el guardado

Made with GAMMA

Vulnerabilidades Comunes

Fallos en la aplicación que permiten ataques y deben ser prevenidos

1

Control de acceso roto: Cuando el sistema te deja entrar a una parte que no te corresponde

2

Desbordamiento de búfer (Buffer Overflow): Se le envía más información de la que la memoria puede manejar, haciendo que el sistema colapse o ejecute órdenes extrañas.

3

Exposición de datos sensibles : Contraseñas guardadas en texto plano, que es cuando las contraseñas no están encriptadas. Se ven a simple vista

4

Configuración insegura (Misconfiguration): Es cuando un sistema o red está mal configurado y deja puertas abiertas sin querer.

Made with GAMMA

Intrusiones y Ataques

Un ataque o una intrusión es cuando alguien accede a un sistema sin permiso, aprovechando errores o debilidades para entrar. ¿Que busca un atacante?:

Robo de Datos

Contraseñas, correos o datos personales filtrados y expuestos

Acceso No Autorizado

Un usuario normal entra al panel de administrador sin permisos

Control del Sistema

El atacante puede borrar o cambiar información crítica

Tipos de Ataques

SQL Injection

El atacante escribe código en formularios para manipular la base de datos. Puede acceder, modificar o borrar información.

Phishing

Engaña al usuario para que entregue sus datos usando mensajes o páginas falsas que parecen reales.

Fuerza Bruta

Prueba muchas contraseñas hasta adivinar la correcta. Funciona mejor si la contraseña es débil.

Made with GAMMA

Seguridad de Red

Protege los datos mientras viajan entre el usuario y el servidor, evitando que sean interceptados o manipulados

Firewall

Filtro que decide qué conexiones entran o salen del sistema, bloqueando tráfico sospechoso

HTTPS

Versión segura de HTTP que cifra la información, evitando que terceros la lean o roben

Protección

Protege datos sensibles como tokens, contraseñas y datos de usuarios

Ejemplo simple: Sin HTTPS → enviar contraseña es como escribirla en una postal (todos la ven). Con HTTPS → es como enviarla en un sobre cerrado (nadie puede leerla).

Made with GAMMA

Errores Comunes en Desarrollo

Confiar en el Frontend

Creer que porque un botón está oculto o un formulario tiene validaciones en HTML/JavaScript, el usuario no puede enviar datos maliciosos. **Todo debe validarse en el backend**.

Hardcodear Secretos

Dejar contraseñas, tokens o claves de API directamente en el código fuente en lugar de usar variables de entorno.

Mensajes de Error Muy Descriptivos

Mostrar al usuario el stack trace completo o el error exacto de la base de datos. Esto le da pistas valiosas a un atacante malicioso.

Librerías Desactualizadas

Ignorar las alertas de seguridad de las dependencias (paquetes de npm o dependencias de Python) que tienen vulnerabilidades conocidas.

Consecuencias de No Proteger

- ### Sin Validación de Inputs

  Abre la puerta a SQL Injection y XSS. Dejas que el usuario dicte qué hace tu servidor.

- ### Sin Protección de Rutas

  Broken Access Control. Cualquiera que adivine la URL puede ejecutar acciones críticas.

- ### Sin HTTPS

  Todo el tráfico viaja en texto plano. Man-in-the-Middle puede leer contraseñas y tokens.

Made with GAMMA

Caso Práctico Real

Escenario: Un usuario accede a información que no le corresponde

¿Qué Falló?

Falló la Autorización (Authorization). El sistema supo quién era el usuario (Autenticación), pero falló en verificar si ese usuario tenía derecho a ver ese recurso.

Es un problema de permisos debido a una vulnerabilidad lógica en el código.

¿Qué es IDOR?

Insecure Direct Object Reference. Un usuario cambia la URL de `tusitio.com/factura/10` a `tusitio.com/factura/11` y el servidor le muestra la factura de otra persona.

¿Cómo Evitarlo como Developer?

01

Validación en Backend

Siempre verificar si el ID del usuario coincide con el ID del propietario del registro

02

Control de Acceso (RBAC)

Implementar sistema de roles estricto y verificar permisos en cada endpoint

03

Usar UUIDs

Identificadores únicos universales en vez de IDs secuenciales para evitar que adivinen URLs

Conceptos Clave

- **JWT (JSON Web Token)**

  Token seguro que el servidor genera en login. Contiene información del usuario y firma que evita modificaciones. Sirve para no pedir login repetidamente.

- **Rate Limiting**

  Controla cuántas solicitudes puede hacer un usuario en un tiempo determinado. Máximo 100 requests por minuto. Protege contra fuerza bruta, spam y sobrecarga del servidor.

Made with GAMMA